
MP6: Stencil + Blending and Fake Shadow

Due Time: Refer to the course web-site

Objectives

Verify that we understand and are able to work with:

- Unity specific skills:
 - There should be no new skills challenged or required
- General linear algebra concepts:
 - Vector line equations and distances travel along a line
 - Vector plane equations
 - Line-Vector intersection
- Shader and relevant general concepts:
 - Working with alpha channel
 - Configuration and control of blending, and, the stencil buffer
 - Applying linear algebra concepts in creating fake shadow geometries

Approach

Recalling that given a direction, $\hat{d}$, projection of a vertex position, V , along this direction can be described by the line:

$$l(t): V + t\hat{d}$$

where the parameter, t , describes the distance travelled along the direction, $\hat{d}$. Now, we know that given a plane with normal, $\hat{n}$, and distance, D , away from the origin, the vector plane equation is:

$$plane: (\hat{n} \cdot p) - D = 0$$

where the parameter, p , is a position on the plane. When the line intersect with the plane:

$$p = V + t\hat{d}$$

Substitute p into the vector plane equation

$$(\hat{n} \cdot (V + t\hat{d})) - D = 0$$

Solve for the only unknown in the equation, t ,

$$t = \frac{D - (\hat{n} \cdot V)}{\hat{n} \cdot \hat{d}}$$

Note that:

- When the denominator, $\hat{n} \cdot \hat{d}$, is close to zero, $\hat{d}$ is almost perpendicular to $\hat{n}$, and there can be no intersection and this it is not possible to project the vertex on the plane.
- When t is less than zero, then, the plane is in the negative direction of $\hat{d}$, and it is again not possible to project the vertex on the plane along $\hat{d}$.

Shadow Casting Lights

Given the linear algebra background, you are to implement projected shadow for MP5. You can assume:

- Light-0: being a directional light and cast shadow based on the direction of this light source.
- Light-1: being a point light and cast shadow from the light position.
 - You can ignore distance attention of point light for this assignment.
- Light-2: being a spotlight and cast shadow based on the position and direction of this light source.

- For Point- and Spot-light: You **must** respect both distance and angular attention where
 - Shadow profile should disappear gradually and
 - Shadow should not exist outside of the distance or angular ranges

You can assume light-0 being directional, light-1 being point, and light-2 being spotlight and will not be changed for MP6. With this assumption, you will demonstrate your understanding of the given projection formulation and knowledge in shader programming by implementing shadows from these three light sources. We will leave Light-3 as a directional (not casting shadow) and use this light source to create basic illumination for examining the shadows.

Objects

All objects must be Phong illuminated by the given four light sources (as in our MP5). Although there are no credits associated with this requirement, each non-Phong illuminated object will result in -10 loss of credit.

Shadow Receiver and Shadow Color

You will allow your users to control the color of the shadow casted, **ShadowColor**. This is not something possible in the physical world, but, our rendering system is *not* attempting to simulate the real world. We will assume there is one global ShadowColor for all light sources.

You can assume one planar receiver in the scene where all fake shadow geometries are only visible on the receiver. The shadowCaster will compute *shadow geometries* and *blend* the geometries as shadow color onto the display window.

- Note, shadow effect is different from object colors, one does not *add* shadow color, instead, one reduces the *strength* of object color when it is in shadow. So, what we really want to do is to *subtract* some percentage of color from the color buffer.
- In our case, recall that there is the *Blend* function, where, we can compute a less than 1.0 alpha value to blend with the color buffer.
- Since each pass is blends the ShadowColor into the color buffer, it effectively blending dark color into the color buffer, effectively reducing the intensity of the object color. In all my passes, assuming the computed shadow contribution is **shadowStrength**, I configured my blending such that:

```
Final color = ShadowColor.rgb * shadowStrength + ColorBuffer.rgb * (1 - shadowStrength) * ShadowColor.a
// Note: a directional light has no distance or angular attention, its shadowStrength is always 1.0
```

Initial Scene Setup

To help me test your programs, please:

- Configure Light-4 as a non-shadow casting directional light, casting lights 90-degrees downward
- Make sure that only Light-4 is switched on initially and all three other shadow-casting lights are switched off.

Hints

- You can use my MP5 solution as the base for implementation. The shader source code is rather large, but, all settings are available.
 - In particular, take a look at the AngularDropOff() and DistanceDropOff() functions in my MP5. You can simply call these functions to determine the proper attenuations.
 - Light information is available in MyLights.cginc.
- I implemented a separate pass for each shadow casting light sources. For this reason, shadow caster will include at least four passes (including the final Phong illumination pass).
 - Z-buffer fighting, remember, each pass is rendering geometries into the same z-buffer. In each separate passes, if you project the same shadow geometry to the exact same distance, then, you may (or will) run into z-buffer depth resolution problem, that, the hardware may not be able to resolve front from back!
 - Fudge the distance to project to by a percentage for each subsequent projections!

- Make sure you understand what I am saying and think about what you can do.
- Important to read and understand [the Unity Blend shader command](#) (similar functionality in GLSL and HLSL)
 - Src: output color from the fragment shader
 - Dst: color already in the color buffer
 - Blend <Color>, <Alpha>
 - Color: Src Dst
 - Alpha: Src Dst
 - E.g., Blend SrcAlpha, DstAlpha, Zero One
 - Color: SrcAlpha DstAlpha → $\text{color} = \text{SrcAlpha} * \text{Source Color} + \text{DskAlpha} * \text{DstColor}$
 - Alpha Zero One → $\text{alpha} = \text{SrcAlpha} * \text{Zero} + \text{DstAlpha} * \text{One}$
- For the convenience of debugging, each of my light can cast different shadow colors, you do not have to implement this. But for senity and convenience of debugging, do consider this.
- **WARNING:** my experience has been that when rgba over or underflows, strange effecnts may occur (blinking on the screen). For this reason, I have a saturate() call before my fragment shader returns.

Credit Distribution

	Credits
1 Unity Scene Setting	
At least 4 objects: 1 receiver + 3 casters	3
All objects are Phong illuminated	
At least 3 light sources with on/off	3
Light type: one each of: Point, Dir, Spot	
4th Non-shadow casting directional light	2
Switched on, color=(1,1,1,1), in -y	
2 Receiver	
Shadow color/alpha specified by user	5
Proper blending of multiple shadows	8
3 Shadow from Directional Light	
On/Off control	2
Able to rotate the light direction	3
Shadow respect light direction	5
Proper blending with Receiver	5
4 Shadow from Point Light	
On/Off Control	2
Able to move light position	3
Shadow respect light position	5
Shadow respect distance attenuation	5
Proper blending with Receiver	5
5 Shadow from Spot Light	
On/Off Control	2
Able to move position and rotate direction	6
Shadow respect position and direction	12
Respect distance and angular attenuation	14
Proper blending with Receiver	5
6 Submission	
Proper folders/unity version	5
Total	100

This assignment counts 10% towards your final grade for this class.